

ICT308 — Projet 8

Supervision Météo & Alerte Agricole en Réseau Plan d'Exécution Détaillé

Cours	ICT308 — Programmation Java Avancée
Durée	10 jours (à partir du 27 juillet 2025)
Équipe	10 étudiants — 4 sous-groupes spécialisés
Thème principal	Multithreading, Pattern Observer, IHM Swing

1. Contexte du Projet

Le projet simule un réseau de capteurs connectés déployés dans une plantation de cacaoyers au Cameroun. Chaque capteur surveille en continu deux indicateurs climatiques critiques : la **température** (°C) et le **taux d'humidité du sol** (%). L'application centralise ces mesures et déclenche automatiquement des alertes visuelles et sonores lorsque l'humidité d'une zone descend sous le seuil critique de **30%**, signalant la nécessité d'activer le système d'irrigation.

2. Architecture Générale — Pattern Observer

L'application repose sur le **Pattern Observer** pour découpler les capteurs de l'interface graphique :

- **CapteurMeteo (Runnable)** : Producteur de données. Tourne dans son propre thread et génère des mesures aléatoires à intervalle régulier. Notifie ses abonnés via l'interface `MeteoListener`.
- **MeteoListener (Interface)** : Contrat d'écoute définissant la méthode `onMesureRecue(double temp, double humidite)`. Toute classe souhaitant recevoir les mesures doit l'implémenter.
- **CentraleMeteo** : Abonné principal. Centralise les données de tous les capteurs, maintient l'historique et orchestre les mises à jour de l'IHM.
- **FenetrePrincipale (Swing)** : Interface utilisateur. Affiche la `JTable` des zones et réagit aux alertes. Ne communique jamais directement avec les capteurs.

Flux de données :

```
Thread CapteurMeteo → onMesureRecue() → CentraleMeteo → SwingUtilities.invokeLater() → JTable
```

3. Décomposition par Équipe

Équipe 1 — Core & Métier (3 étudiants)

Responsable de toute la logique métier et de l'architecture du pattern Observer.

- Interface `MeteoListener` avec méthode `onMesureRecue(double temp, double humidite)`
- Enum `Zone` : `ZONE_A`, `ZONE_B`, `ZONE_C`, `ZONE_D`, `ZONE_E` (5 zones de plantation)
- Classe `MesureMeteo (Serializable)` : `nomZone`, `temperature`, `humidite`, `timestamp`

- Classe CapteurMeteo implements Runnable : génère temp [15°–40°C] et humidité [15%–85%] via Math.random(), notifie les listeners toutes les 2–3 secondes
- Classe CentraleMeteo : liste des capteurs, liste des listeners, méthodes abonner() / desabonner()

Algorithme clé du capteur :

```
while (!Thread.interrupted()) { double temp = 15 + Math.random() * 25; double humidite = 15 + Math.random() * 70; notifierListeners(temp, humidite); Thread.sleep(2000 + (long)(Math.random() * 1000)); }
```

Équipe 2 — Persistance & Données (2 étudiants)

Responsable de la sauvegarde et de la traçabilité des mesures.

- Sérialisation Java : sauvegarde périodique de List dans historique_meteo.ser
- Méthodes sauvegarder(List) et charger() → List
- Export CSV : rapport_alertes.csv généré à chaque déclenchement d'alerte
- Format CSV : Zone, Temperature, Humidite, Timestamp, Statut
- Fermeture propre des ObjectOutputStream via bloc finally (prévention de fuites)

Équipe 3 — IHM Swing (3 étudiants)

Responsable de toute l'interface graphique et de la mise en forme visuelle des alertes.

- FenetrePrincipale (JFrame, BorderLayout) : NORTH = titre + horloge | CENTER = JTable | SOUTH = statistiques
- MeteoTableModel extends AbstractTableModel : colonnes Zone | Temp (°C) | Humidité (%) | Statut
- MeteoTableCellRenderer : colore la ligne en rouge (Color.RED) si humidité < 30%
- Panneau SOUTH : moyenne globale température, moyenne humidité, nombre d'alertes actives
- Signal sonore : Toolkit.getDefaultToolkit().beep() à chaque nouvelle alerte
- Signal visuel : JLabel clignotant 'ALERTE IRRIGATION REQUISE' visible si ≥ 1 zone en alerte
- Boutons : Démarrer les capteurs / Arrêter les capteurs / Exporter CSV

Équipe 4 — Performance & Multithreading (2 étudiants)

Responsable de la gestion des threads, de la synchronisation et de la réactivité de l'IHM.

- Lancement de 5 threads CapteurMeteo (un par Zone), via ScheduledExecutorService ou Thread
- RÈGLE ABSOLUE : toutes les mises à jour Swing via SwingUtilities.invokeLater()
- Liste des mesures protégée : Collections.synchronizedList(new ArrayList<>())
- Arrêt propre des threads : bouton 'Arrêter' appelle thread.interrupt() sur chaque capteur
- Aucun deadlock : pas de synchronized croisé entre capteur et IHM
- Test de charge : simulation de 10 capteurs simultanés sans dégradation des performances

4. Planning sur 10 Jours

Jours	Phase	Livrable attendu	Équipe
J1 – J2	Setup & Métier	MeteoListener, CapteurMeteo, MesureMeteo compilent et testés	Équipe 1 & 2
J3 – J4	IHM de base	Fenêtre Swing avec JTable affichant des données statiques (mock)	Équipe 3

Jours	Phase	Livrable attendu	Équipe
J5 – J6	Intégration threads	Capteurs actifs, JTable se met à jour en temps réel sans gel	Eq.4 + Eq.1
J7	Alertes visuelles	Coloration rouge + beep fonctionnels au passage sous 30%	Eq.3 + Eq.4
J8	Persistance	Sauvegarde .ser et export CSV opérationnels	Eq.2
J9	Tests & Robustesse	Aucun deadlock, aucun crash, tests d'arrêt/redémarrage	Tous
J10	Soutenance	Chaque membre explique sa contribution précisément	Individuel

5. Points Critiques à Ne Pas Rater

■ RÈGLE EDT (Event Dispatch Thread)

Toute modification d'un composant Swing doit se faire dans l'EDT. Utiliser OBLIGATOIREMENT `SwingUtilities.invokeLater()` -> { ... } pour chaque appel à `fireTableDataChanged()` ou `repaint()` depuis un thread capteur.

■ Seuil d'alerte = 30% d'humidité

La condition doit être explicite dans le `TableCellRenderer` : `if (humidite < 30.0) { setBackground(Color.RED); }`. Ne pas oublier le `else { setBackground(UIManager.getColor("Table.background")); }` pour réinitialiser les cellules normales.

■ Pattern Observer strict

Le CapteurMeteo ne doit JAMAIS avoir de référence directe à la FenetrePrincipale. Il communique uniquement via l'interface `MeteoListener`. Cela garantit le découplage et facilite les tests unitaires.

■ Synchronisation des collections

La liste partagée de mesures doit être thread-safe. Utiliser `Collections.synchronizedList()` ou `CopyOnWriteArrayList` pour éviter les `ConcurrentModificationException`.

■ Arrêt propre des threads

Utiliser `thread.interrupt()` et tester `Thread.interrupted()` dans la boucle du capteur. Ne jamais appeler `Thread.stop()` (déprécié et dangereux).

■ Fermeture des flux de persistance

Toujours fermer `ObjectOutputStream` dans un bloc `finally` ou utiliser `try-with-resources` pour éviter la corruption du fichier .ser.

6. Barème Cibl   (20 Points)

Crit��re	Points	Ce qui est ��valu��	Strat��gie
Architecture & POO	14 pts	Interface <code>MeteoListener</code> , h��ritage propre, Pattern Observer (priv��)	Capteurs Observer (priv��) s��par��, pas de r��f��rence directe IHM ↔

Critère	Points	Ce qui est évalué	Stratégie
Interface Swing	/4 pts	JTable dynamique, coloration rouge < 30%	Boite à outils principal + AbstractTableModel personnalisé
Multithreading	/3 pts	5 threads stables, pas de deadlock, EDT	SwingUtilities.invokeLater() systématique, collections synchronisées
Persistance	/3 pts	Sérialisation .ser sans erreur, fermeture propre des flux	proprietés des sources, test de chargement au démarrage
Robustesse	/2 pts	Aucun crash sur arrêt brutal, gestion des interruptions	Thread.sleep(), InterruptedException
Soutenance	/4 pts	Chaque étudiant explique sa classe et ses contributions	Chaque membre prépare 2 min d'explication de sa contribution

7. Structure des Classes — Vue d'Ensemble

MeteoListener	<i>interface</i>	onMesureRecue(double temp, double humidite) : void
Zone	<i>enum</i>	ZONE_A, ZONE_B, ZONE_C, ZONE_D, ZONE_E
MesureMeteo	<i>class implements Serializable</i>	nomZone : String, temperature : double, humidite : double, timestamp : LocalDateTime
CapteurMeteo	<i>class implements Runnable</i>	nomZone : String, listeners : List, run() : void, ajouterListener() : void
CentraleMeteo	<i>class implements MeteoListener</i>	capteurs : List, historique : List [synchronized], onMesureRecue() : void
PersistanceService	<i>class</i>	sauvegarder(List) : void, charger() : List, exporterCSV() : void
MeteoTableModel	<i>class extends AbstractTableModel</i>	donnees : List, getValueAt() : Object, fireTableDataChanged() : void
MeteoTableCellRenderer	<i>class extends DefaultTableCellRenderer</i>	getTableCellRendererComponent() : met fond rouge si humidite < 30
FenetrePrincipale	<i>class extends JFrame implements MeteoListener</i>	tableModel, btnDemarrer, btnArreter, lblAlerte, onMesureRecue() via invokeLater()
Main	<i>class</i>	Point d'entrée : crée CentraleMeteo + FenetrePrincipale, abonne la fenêtre

Ce document est le plan d'exécution officiel du Projet 8 (ICT308). Chaque étudiant doit maîtriser sa contribution et être capable de l'expliquer individuellement lors de la soutenance finale. La note de soutenance individuelle représente 4/20 points.